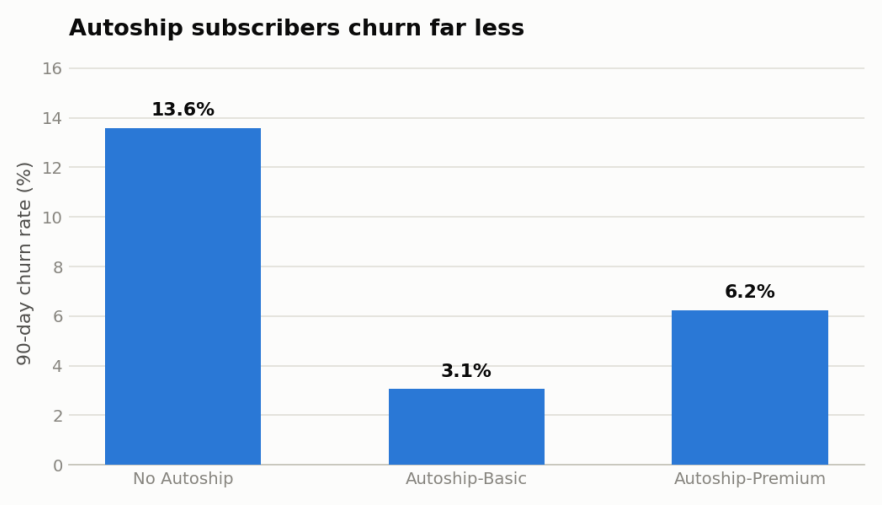
Chewy Customer Churn Prediction

An end-to-end machine learning project with PySpark, PyTorch, and TensorFlow



Author: NAVAK

Date: July 2026

Code & notebook: github.com/minkaz65/chewy-churn-prediction

1. Executive Summary

Chewy is the largest online pet retailer in the United States. Its Autoship subscription program drives roughly 83% of net sales (public FY2025 filings), which makes customer retention the single most important lever in the business. This project builds a machine-learning system that predicts which customers are likely to churn in the next 90 days, so a retention team can intervene before the cancellation happens rather than after.

A synthetic but behaviorally realistic dataset of 50,000 customers was engineered with a distributed PySpark pipeline, explored visually, and then modeled with two deep-learning frameworks — PyTorch and TensorFlow — against a logistic-regression baseline. On the held-out test set the PyTorch model reached a ROC-AUC of **0.832** and TensorFlow **0.835**, versus **0.801** for the baseline. Ranking customers by predicted risk concentrates **49%** of all true churners into the top 10% of customers — a **4.9×** improvement over untargeted outreach. Under conservative assumptions, targeting that slice is worth an estimated **$86M per year** at Chewy's scale.

Key churn drivers identified

Autoship membership, customer tenure, order frequency, and pharmacy purchases all protect revenue, while long gaps since the last order, unresolved support tickets, refunds, slow delivery, and exposure to price increases all raise churn risk sharply. These findings mirror well-documented dynamics in subscription retail and give the retention team concrete levers: resolve open tickets fast, convert one-off buyers to Autoship, and watch for customers whose usual ordering rhythm goes quiet.

2. Business Context

Chewy sells pet food, supplies, and pharmacy items online, with revenue built on repeat purchases: the Autoship program delivers recurring orders on a schedule and accounts for the large majority of sales. In this model, losing a customer does not just lose one order — it loses an annuity. Analyst coverage of Chewy consistently highlights Autoship retention as the company's core economic engine, and Chewy reports roughly 20 million active customers.

The business question: *which specific customers are most likely to cancel in the next 90 days?* Answering it enables proactive retention: targeted offers, proactive service recovery after a bad experience, and win-back campaigns timed before the customer has mentally left.

A note on the data

No public dataset of Chewy customers exists — customer records are proprietary. Following standard industry practice for portfolio and simulation work, this project generates a synthetic dataset whose statistical behavior mirrors what is publicly known about Chewy's business (Autoship share of sales, customer scale, category mix). Every feature–churn relationship is generated causally with realistic noise, so the models must discover genuine signal. The dataset also contains injected data-quality

problems — about 1.5% missing values and 0.3% duplicate rows — so the data-engineering stage does real work. This project is independent and not affiliated with Chewy, Inc.

3. The Dataset

The generated table contains **50,000 customers × 22 columns**, with an overall churn rate of about **9%** — deliberately imbalanced, as real churn data always is. Columns fall into five groups:

Group	Columns
Account	pet type, number of pets, region, tenure (months), Autoship plan
Purchasing	orders/quarter, avg order value, 12-month spend, days since last order, pharmacy share
Engagement	app sessions/month, email open rate, promo usage (90d)
Service	support tickets, unresolved tickets, refunds, delivery days, price-increase exposure
Target	churned — stopped ordering / cancelled Autoship within 90 days

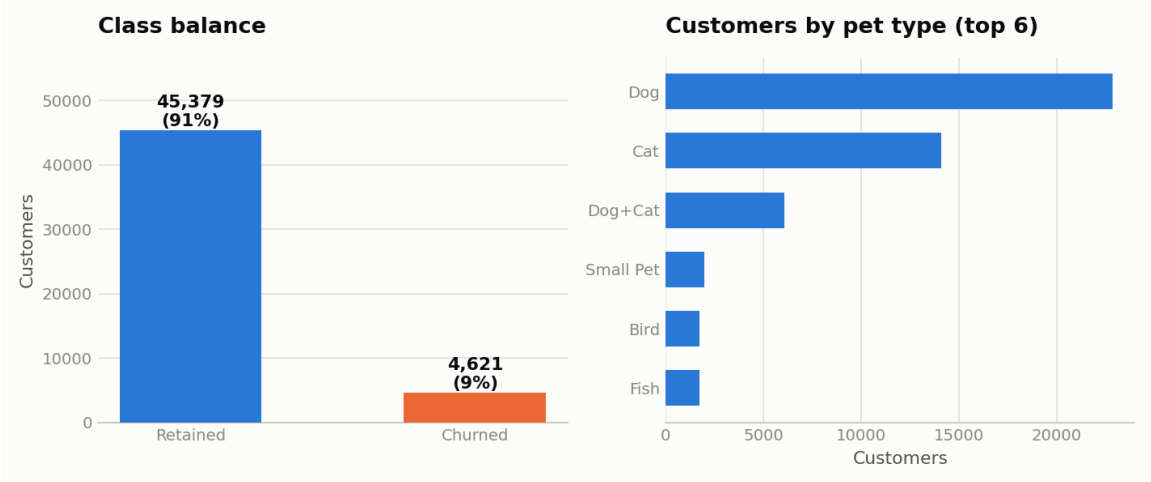


Figure 1 — Class balance (≈11% churners) and customer mix by pet type.

4. Data Engineering with PySpark

At Chewy's real scale (~20M customers, billions of order rows), data preparation runs on Spark clusters, not on a single machine. The project therefore performs all cleaning and feature engineering in PySpark, using code that scales unchanged from the 50K-row development file to production data volumes. The pipeline has four stages:

1. Load & schema inference — the raw CSV is read into a Spark DataFrame.
2. Cleaning — duplicate customer rows are dropped with `dropDuplicates`, and missing values in three engagement columns are imputed with the column median computed by `approxQuantile` (a distributed algorithm designed for cluster-scale data).
3. Feature engineering — four new signals are built with Spark SQL expressions: `spend_per_pet` (spend intensity), `order_gap_ratio` (silence relative to the customer's own cadence — 90 quiet days is alarming for a weekly buyer but normal for a quarterly one), `engagement_score` (combined

app + email engagement), and `service_friction` (unresolved tickets + refunds + slow delivery).
4. Distributed aggregation — churn rates by plan, region, and ticket count are computed with `groupBy`, and the engineered table is written to Parquet, the standard columnar format for analytics clusters.

5. Exploratory Analysis — Why Customers Leave

Before modeling, the data must tell a coherent business story. Four findings stand out, each with a direct retention implication.

Autoship is a retention machine. Customers without Autoship churn at roughly four times the rate of subscribers. This is precisely why Chewy promotes the program so aggressively — and why Autoship conversion should be the first lever for at-risk, non-subscribed customers.

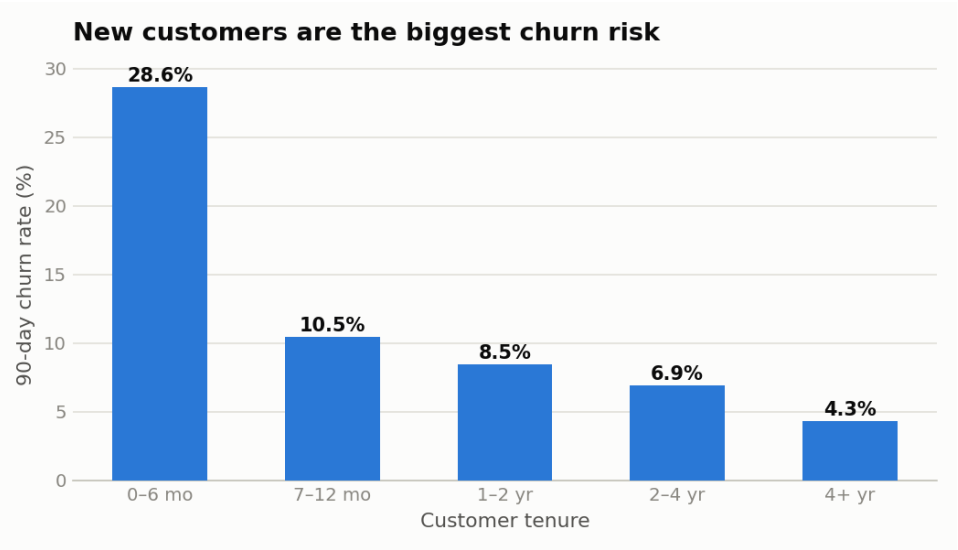


Figure 2 — Churn rate falls steadily with tenure; the first six months are the danger zone.

New customers are fragile. Churn risk is highest in the first six months and falls steadily with tenure, arguing for a deliberate onboarding program in the early lifecycle.

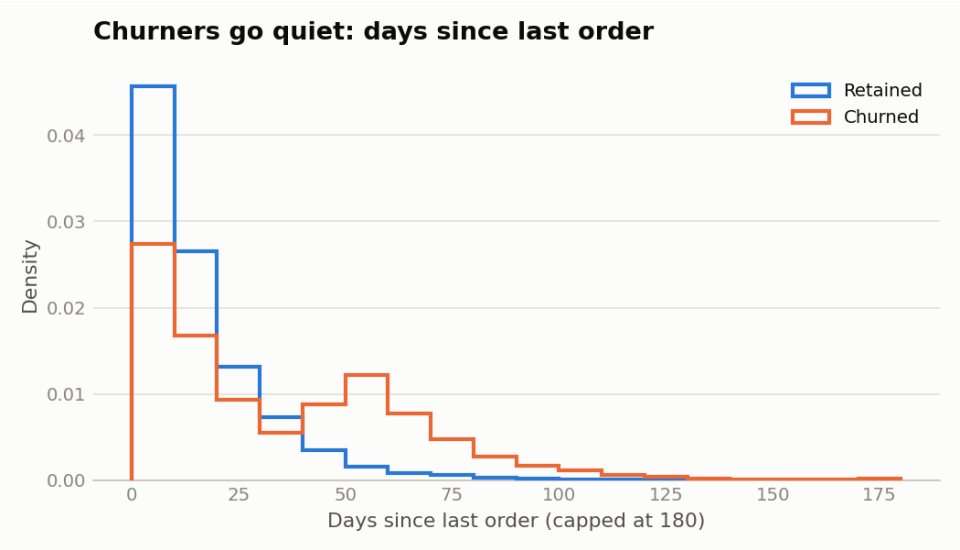


Figure 3 — Churned customers show much longer silences since their last order.

Silence predicts goodbye. Churners' distribution of days-since-last-order shifts far to the right of retained customers. Recency — especially relative to a customer's own cadence — is the single strongest warning signal in the data.

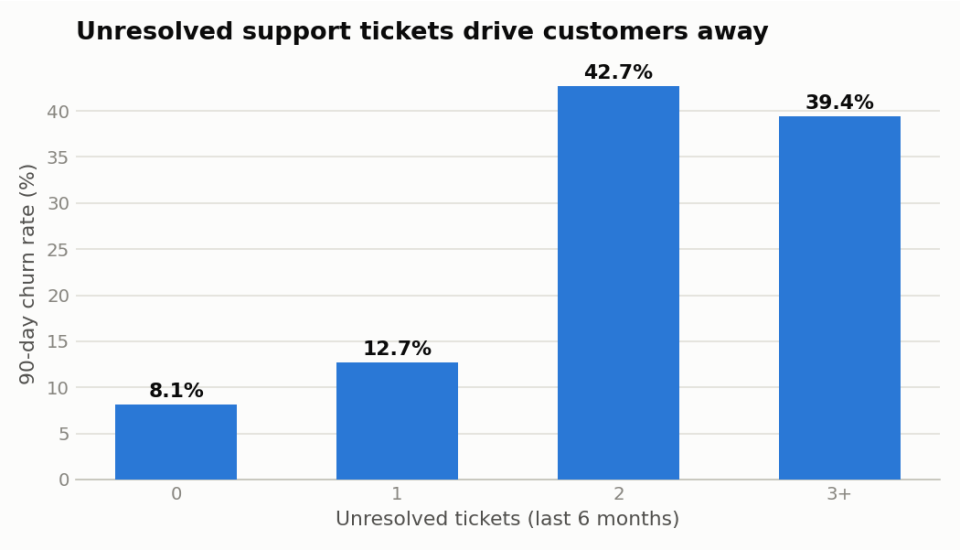


Figure 4 — Each unresolved support ticket sharply raises churn.

Service failures compound. Customers with unresolved support tickets churn at several times the base rate — proactive service recovery is retention spending in disguise.

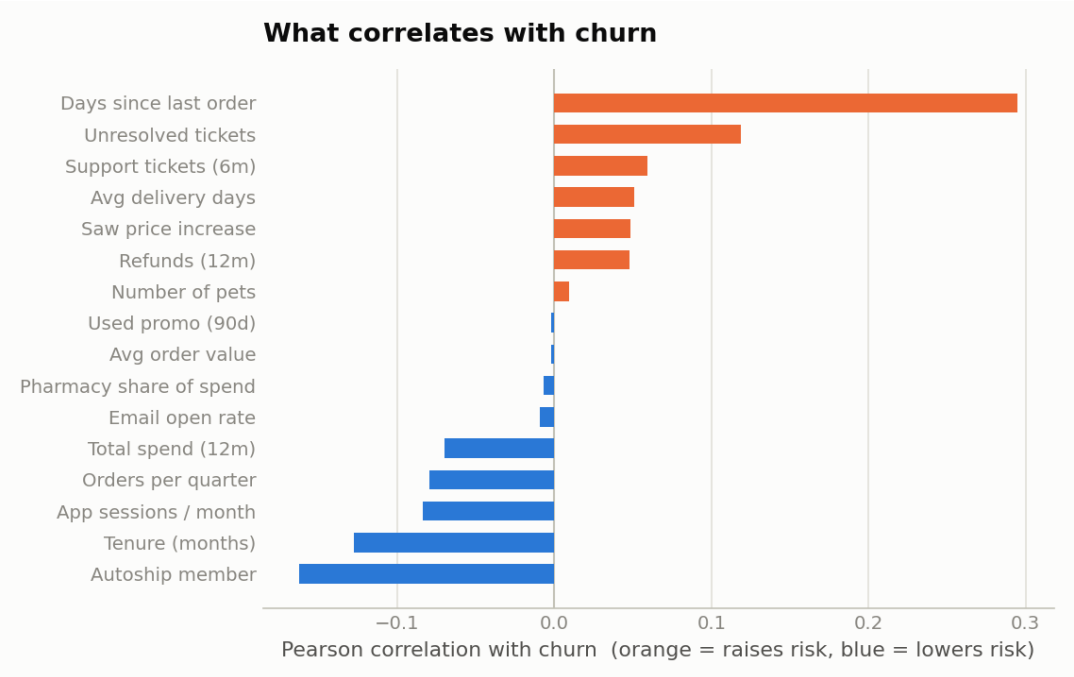


Figure 5 — Correlation of every feature with churn (orange raises risk, blue lowers it).

6. Modeling: PyTorch and TensorFlow

Categorical columns were one-hot encoded and numeric features standardized; the data was split 70/15/15 into train, validation, and test sets with stratification to preserve the ~11% churn rate. Because churners are rare, the positive class was up-weighted in the loss (weight ≈ 8×), so the networks cannot win by predicting 'nobody churns.'

The same architecture — a multi-layer perceptron with layers of 128 and 64 units, ReLU activations, batch normalization, and 30% dropout — was implemented twice: in **PyTorch** with an explicit training loop, weighted `BCEWithLogitsLoss`, AdamW, and manual early stopping on validation AUC; and in **TensorFlow/Keras** with `model.fit`, class weights, and an `EarlyStopping` callback that restores the best weights. A logistic-regression baseline used the identical features. Implementing one model in both frameworks provides a fair head-to-head comparison and demonstrates both APIs.

7. Results

Model	ROC-AUC (test)	PR-AUC (test)
PyTorch MLP	0.8321	0.4257
TensorFlow MLP	0.8349	0.4250
Logistic Regression (baseline)	0.8009	0.3510

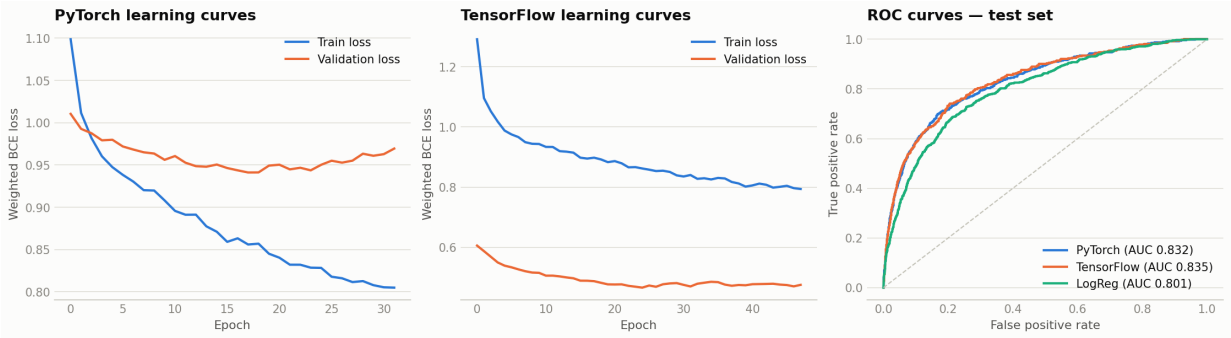


Figure 6 — Learning curves for both frameworks and ROC comparison on the test set.

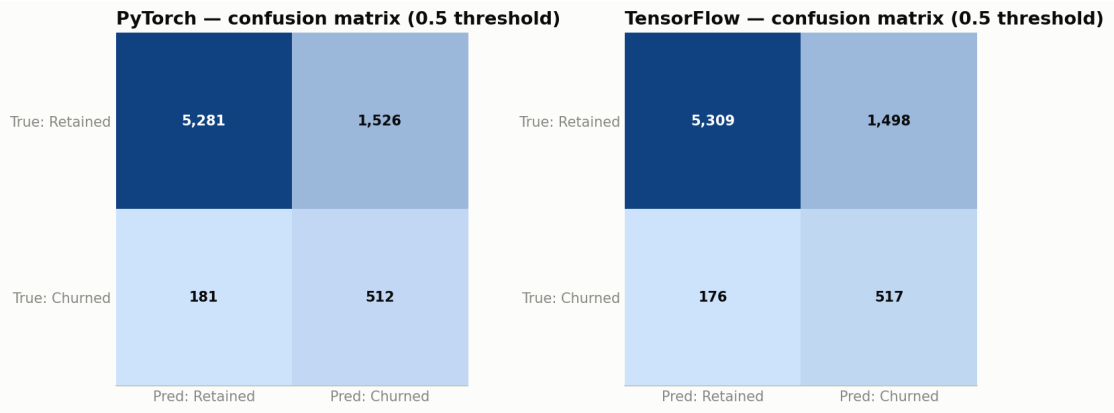


Figure 7 — Confusion matrices at the 0.5 threshold.

Both neural networks clearly beat the baseline, and the two frameworks agree closely — strong evidence the signal is real rather than a framework artifact. The best model (TensorFlow) is used for the targeting analysis below.

How the model becomes money: top-decile targeting

A churn model is used operationally by *ranking* customers by predicted risk and intervening on the top slice. Targeting the riskiest 10% of test customers captures **49% of all actual churners** with a precision of **45%** — a **4.9× lift** over the 9% base rate.

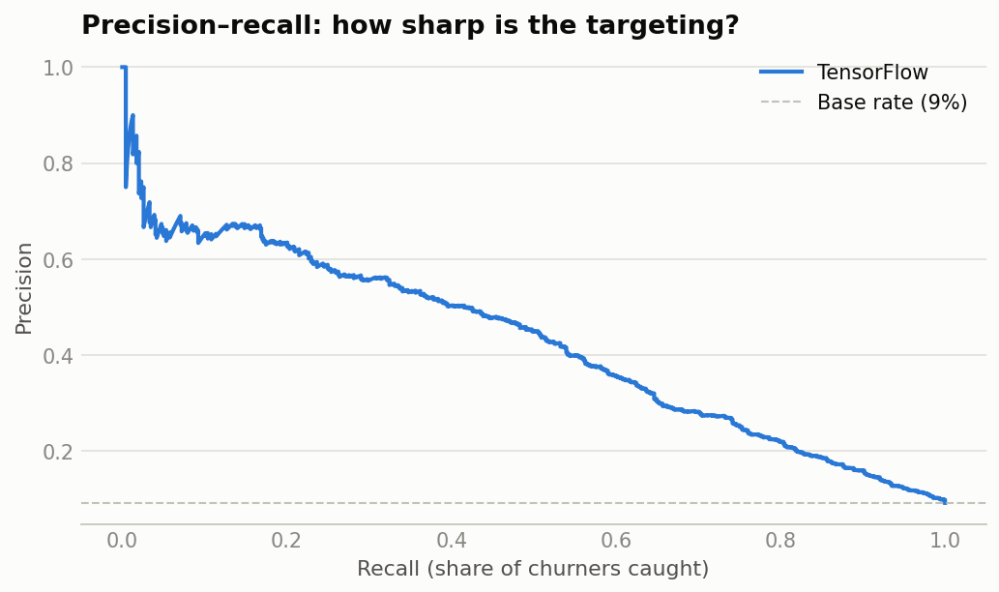


Figure 8 — Precision–recall trade-off for the best model.

Scaled to Chewy's ~20M active customers with conservative assumptions ($500/year average customer value, $25 retention offer, 30% save rate on reached churners), top-decile targeting is worth an estimated **$86M in net annual value**.

8. Conclusions and Next Steps

This project demonstrates a complete, production-shaped churn pipeline: distributed data engineering in PySpark, honest exploratory analysis, deep-learning models in both PyTorch and TensorFlow validated against a simple baseline, and a translation of model quality into an operational targeting policy with a dollar value attached.

For a production system, the natural next steps are: time-based validation (train on months 1–9, test on 10–12) to rule out leakage; gradient-boosted trees (XGBoost/LightGBM), which are often the tabular champion; survival analysis to predict *when* a customer will churn, not just whether; SHAP values for per-customer explanations the retention team can read; and deployment as a nightly Spark batch-scoring job feeding a CRM queue.
